

1. PROJECT OVERVIEW

- **Project Title:** ComicCraft - AI Comic Story Creator using Gemini Models

2. ARCHITECTURAL OVERVIEW The architecture of ComicCraft follows a modern, decoupled three-tier system: the Frontend (Client Presentation Layer), the Backend (Application Logic Layer), and the External Services Layer (AI APIs). This separation of concerns ensures that the application is highly modular, easily maintainable, and scalable.

3. FRONTEND PRESENTATION LAYER

- **Technologies Used:** HTML5, CSS3, Jinja2 Templates.
- **Functionality:** This layer is responsible for client interaction. It consists of `index.html` (for capturing user inputs like Prompt, Tone, and Style), `comic_preview.html` (for dynamically displaying the AI-generated panels using Jinja2 loops), and `export_success.html` (for confirming PDF downloads).

4. BACKEND APPLICATION LAYER

- **Framework:** FastAPI (Python).
- **Server:** Uvicorn ASGI Server.
- **Functionality:** The backend acts as the central brain of the application. It receives POST requests from the frontend and coordinates the entire workflow. The primary routing logic is handled inside `routes.py`.
- **Modular Components:**
 - `gemini_flash.py`: Handles the HTTP request to the Gemini API to fetch the JSON structural outline.
 - `gemini_pro.py`: Manages the generation of detailed character dialogues and narrations.
 - `image_generator.py`: Executes asynchronous requests to the Hugging Face Stable Diffusion API to fetch panel illustrations. Includes robust exception handling to bypass network restrictions (Errno

11001) or HTTP 402/500 errors by deploying fallback image generation.

- `layout_builder.py`: A utility module that merges the output strings from the Gemini models with the saved image paths into a cohesive dictionary.
- `exporters.py`: Utilizes the FPDF module to dynamically calculate coordinates and render the text and images onto a multi-page PDF document.

5. EXTERNAL AI SERVICES LAYER

The application heavily relies on cloud-based Generative AI models. The backend communicates with these external endpoints via secure HTTPS protocols using `urllib.request`.

- **Google Generative AI APIs**: For text and JSON generation.
- **Hugging Face Inference API**: For Stable Diffusion image generation.
- **Security**: API keys are injected into the application securely during runtime using the `python-dotenv` library, ensuring credentials are never exposed to the frontend or version control.

6. DATA STORAGE AND FILE MANAGEMENT

Unlike traditional web apps, ComicCraft currently focuses on stateless generation.

- **Static Storage**: Generated `.jpg` illustrations are stored temporarily in the `static/panels/` directory.
- **Export Storage**: Compiled `.pdf` files are stored in the `static/exports/` directory before being sent to the client for download.

7. SYSTEM INTERACTION FLOW (SEQUENCE)

1. Browser -> `routes.py` (POST /generate)
2. `routes.py` -> `gemini_flash.py` (Returns JSON Outline)
3. `routes.py` -> `gemini_pro.py` (Returns Full Story)

4. `routes.py` -> `image_generator.py` (Returns Image Paths)
5. `routes.py` -> `layout_builder.py` (Returns structured list)
6. `routes.py` -> `exporters.py` (Generates PDF)
7. `routes.py` -> `comic_preview.html` (Renders Final Output)